

NAME: S. Harshavardhan

REG:192524106

COURSE:CSA1718-AI

Experiment 1: Depth First Search (DFS)

Aim

To implement the **Depth First Search (DFS)** algorithm in Python to traverse a graph.

Algorithm

1. Create a graph using an adjacency list.
2. Create an empty set to store visited nodes.
3. Start DFS from the source node.
4. Mark the current node as visited and display it.
5. Recursively visit all unvisited neighboring nodes.
6. Repeat until all reachable nodes are visited.
7. Stop when there are no more unvisited adjacent nodes.

Procedure

1. Define the graph as an adjacency list.
2. Initialize an empty visited set.
3. Create a recursive function dfs().
4. Check whether the current node is visited.
5. If not visited, print the node and mark it as visited.
6. Recursively visit each adjacent node.
7. Execute the DFS function from the starting node.

Python Code

Depth First Search (DFS)

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],
```

```

'C': ['F'],
'D': [],
'E': ['G'],
'F': [],
'G': []
}

```

```
visited = set()
```

```
def dfs(node):
```

```
    if node not in visited:
```

```
        print(node, end=" ")
```

```
        visited.add(node)
```

```
    for neighbour in graph[node]:
```

```
        dfs(neighbour)
```

```
print("DFS Traversal:")
```

```
dfs('A')
```

Output

The screenshot shows a Python3 code editor with the following code:

```

1 graph = {
2     'A': ['B', 'C'],
3     'B': ['D', 'E'],
4     'C': ['F'],
5     'D': [],
6     'E': ['G'],
7     'F': [],
8     'G': []
9 }
10
11 visited = set()
12
13 def dfs(node):
14     if node not in visited:
15         print(node, end=" ")
16         visited.add(node)
17
18     for neighbour in graph[node]:
19         dfs(neighbour)
20
21 # Start DFS from node A
22 print("DFS Traversal:")
23 dfs('A')

```

The right sidebar shows the execution status and output:

Status : Successfully executed

Time: 0.0100 secs

Memory: 8.496 Mb

Your Output

```
DFS Traversal:
A B D E G C F
```

DFS Traversal:

A B D E G C F

Result

Thus, the Depth First Search (DFS) algorithm was successfully implemented in Python and the graph was traversed using DFS.

Experiment 2: Breadth First Search (BFS)

Aim

To implement the **Breadth First Search (BFS)** algorithm in Python to traverse a graph.

Algorithm

1. Create a graph using an adjacency list.
2. Create an empty queue and visited set.
3. Insert the starting node into the queue.
4. Mark the starting node as visited.
5. Remove the front node from the queue and display it.
6. Add all unvisited neighboring nodes to the queue.
7. Repeat until the queue becomes empty.

Procedure

1. Define the graph as an adjacency list.
2. Import deque from the collections module.
3. Initialize an empty queue and visited set.
4. Insert the starting node into the queue.
5. Remove nodes one by one from the queue.
6. Print each visited node and add its unvisited neighbors to the queue.
7. Continue until the queue is empty.

Python Code

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['G'],  
    'F': [],  
    'G': []  
}
```

```
def bfs(start):  
    visited = set()  
    queue = deque([start])  
    visited.add(start)  
  
    while queue:  
        node = queue.popleft()  
        print(node, end=" ")  
  
        for neighbour in graph[node]:  
            if neighbour not in visited:  
                visited.add(neighbour)  
                queue.append(neighbour)  
  
print("BFS Traversal:")  
bfs('A')
```

Output

BFS Traversal:

```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/SANJAI/OneDrive/Desktop/8 puzzle program.py =====
A B C D E F
>>>

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = []
queue = []

def bfs(node):
    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print(s, end=" ")

        for i in graph[s]:
            if i not in visited:
                visited.append(i)
                queue.append(i)

bfs('A')
```

A B C D E F G

Result

Thus, the Breadth First Search (BFS) algorithm was successfully implemented in Python and the graph was traversed using BFS.

These are in a standard **record/lab manual format**, ready to upload or write in your AI laboratory record.